Add data tests to your DAG

> **TIP**
>
> Use dbt Copilot, available for dbt Enterprise and Enterprise+ accounts, to generate data tests in the Studio IDE only.

Related reference docs

- Test command
- Data test properties
- Data test configurations
- Test selection examples

> **IMPORTANT**
>
> Tests are now called data tests to disambiguate from unit tests. The YAML key `tests:` is still supported as an alias for `data_tests:`. Refer to New `data_tests:` syntax for more information.

Overview

Data tests are assertions you make about your models and other resources in your dbt project (for example, sources, seeds, and snapshots). When you run `dbt test`, dbt will tell you if each test in your project passes or fails.

You can use data tests to improve the integrity of the SQL in each model by making assertions about the results generated. Out of the box, you can test whether a specified column in a model only contains non-null values, unique values, or values

that have a corresponding value in another model (for example, a `customer_id` for an `order` corresponds to an `id` in the `customers` model), and values from a specified list. You can extend data tests to suit business logic specific to your organization – any assertion that you can make about your model in the form of a select query can be turned into a data test.

Data tests return a set of failing records. Generic data tests (also known as schema tests) are defined using `test` blocks.

Like almost everything in dbt, data tests are SQL queries. In particular, they are `select` statements that seek to grab "failing" records, ones that disprove your assertion. If you assert that a column is unique in a model, the test query selects for duplicates; if you assert that a column is never null, the test seeks nulls. If the data test returns zero failing rows, it passes, and your assertion has been validated.

There are two ways of defining data tests in dbt:

- A **singular** data test, in its simplest form, is when you write a SQL query that returns failing rows, you can save that query in a `.sql` file within your test directory. It's now a data test, and it will be executed by the `dbt test` command.
- A **generic** data test is a parameterized query that accepts arguments. The test query is defined in a special `test` block (like a macro). Once defined, you can reference the generic test by name throughout your `.yml` files—define it on models, columns, sources, snapshots, and seeds. dbt ships with four generic data tests built in, and we think you should use them!

Defining data tests is a great way to confirm that your outputs and inputs are as expected, and helps prevent regressions when your code changes. Because you can use them over and over again, making similar assertions with minor variations, generic data tests tend to be much more common—they should make up the bulk of your dbt data testing suite. That said, both ways of defining data tests have their time and place.

> **CREATING YOUR FIRST DATA TESTS**
>
> If you're new to dbt, we recommend that you check out our online dbt Fundamentals course or quickstart guide to build your first dbt project with models and tests.

Singular data tests

The simplest way to define a data test is by writing the exact SQL that will return failing records. We call these "singular" data tests, because they're one-off assertions usable for a single purpose.

These tests are defined in `.sql` files, typically in your `tests` directory (as defined by your `test-paths` config). **Note:** The `tests/` directory (`test-paths`) is reserved for singular and generic data tests (SQL). Unit test YAML definitions must live under your project's `model-paths` (for example, in the `models/` directory), not in `tests/` . You can use Jinja (including `ref` and `source`) in the test definition, just like you can when creating models. Each `.sql` file contains one `select` statement, and it defines one data test:

tests/assert_total_payment_amount_is_positive.sql

```
-- Refunds have a negative amount, so the total amount should always be >= 0.
-- Therefore return records where total_amount < 0 to make the test fail.
select
    order_id,
    sum(amount) as total_amount
from {{ ref('fct_payments') }}
group by 1
having total_amount < 0
```

The test name is the file name: `assert_total_payment_amount_is_positive` .

Note:

- Omit semicolons (;) at the end of the SQL statement in your singular test files, as they can cause your data test to fail.
- Singular data tests placed in the tests directory are automatically executed when running `dbt test` . Don't reference singular tests in `model_name.yml` , as they are not treated as generic tests or macros, and doing so will result in an error.

To add a description to a singular data test in your project, add a `.yml` file to your `tests` directory, for example, `tests/schema.yml` with the following content:

tests/schema.yml

```
data_tests:
```

```
  - name: assert_total_payment_amount_is_positive
    description: >
      Refunds have a negative amount, so the total amount should always be >=
0.
      Therefore return records where total amount < 0 to make the test fail.
```

Singular data tests are so easy that you may find yourself writing the same basic structure repeatedly, only changing the name of a column or model. By that point, the test isn't so singular! In that case, we recommend generic data tests.

Generic data tests

Certain data tests are generic: they can be reused over and over again. A generic data test is defined in a `test` block, which contains a parameterized query and accepts arguments. It might look like:

```
{% test not_null(model, column_name) %}

    select *
    from {{ model }}
    where {{ column_name }} is null

{% endtest %}
```

You'll notice that there are two arguments, `model` and `column_name`, which are then templated into the query. This is what makes the data test "generic": it can be defined on as many columns as you like, across as many models as you like, and dbt will pass the values of `model` and `column_name` accordingly. Once that generic test has been defined, it can be added as a *property* on any existing model (or source, seed, or snapshot). These properties are added in `.yml` files in the same directory as your resource.

> **INFO**
>
> If this is your first time working with adding properties to a resource, check out the docs on declaring properties.

Out of the box, dbt ships with four generic data tests already defined: `unique`, `not_null`, `accepted_values`, and `relationships`. Here's a full example using those tests on an `orders` model:

```yaml
models:
  - name: orders
    columns:
      - name: order_id
        data_tests:
          - unique
          - not_null
      - name: status
        data_tests:
          - accepted_values:
              arguments: # available in v1.10.5 and higher. Older versions can set the <argument_name> as the top-level property.
                values: ['placed', 'shipped', 'completed', 'returned']
      - name: customer_id
        data_tests:
          - relationships:
              arguments:
                to: ref('customers')
                field: id
```

In plain English, these data tests translate to:

- `unique`: the `order_id` column in the `orders` model should be unique
- `not_null`: the `order_id` column in the `orders` model should not contain null values
- `accepted_values`: the `status` column in the `orders` model should be one of `'placed'`, `'shipped'`, `'completed'`, or `'returned'`
- `relationships`: each `customer_id` in the `orders` model exists as an `id` in the `customers` table (also known as referential integrity)

Behind the scenes, dbt constructs a `select` query for each data test, using the parameterized query from the generic test block. These queries return the rows where your assertion is *not* true; if the test returns zero rows, your assertion passes.

You can find more information about these data tests, and additional configurations (including `severity` and `tags`) in the reference section. You can also add descriptions to the Jinja macro that provides the core logic of a generic data test. Refer to the Add description to generic data test logic for more information.

More generic data tests

Those four tests are enough to get you started. You'll quickly find you want to use a wider variety of data tests — a good thing! You can also install generic data tests from a package, or write your own, to use (and reuse) across your dbt project. Check out the guide on custom generic data tests for more information.

> **INFO**
>
> There are generic data tests defined in some open-source packages, such as dbt-utils and dbt-expectations — skip ahead to the docs on packages to learn more!

Example

To add a generic (or "schema") data test to your project:

1. Add a `.yml` file to your `models` directory, for example, `models/schema.yml`, with the following content (you may need to adjust the `name:` values for an existing model)

models/schema.yml

```yaml
models:
  - name: orders
    columns:
      - name: order_id
        data_tests:
          - unique
          - not_null
```

2. Run the `dbt test` command:

```
$ dbt test

Found 3 models, 2 tests, 0 snapshots, 0 analyses, 130 macros, 0 operations, 0
seed files, 0 sources

17:31:05 | Concurrency: 1 threads (target='learn')
```

```
17:31:05 |
17:31:05 | 1 of 2 START test not_null_order_order_id.....................
[RUN]
17:31:06 | 1 of 2 PASS not_null_order_order_id...........................
[PASS in 0.99s]
17:31:06 | 2 of 2 START test unique_order_order_id.......................
[RUN]
17:31:07 | 2 of 2 PASS unique_order_order_id.............................
[PASS in 0.79s]
17:31:07 |
17:31:07 | Finished running 2 tests in 7.17s.

Completed successfully

Done. PASS=2 WARN=0 ERROR=0 SKIP=0 TOTAL=2
```

3. Check out the SQL dbt is running by either:
   - **dbt:** checking the Details tab.
   - **dbt Core:** checking the `target/compiled` directory

Unique test

Compiled SQL | Templated SQL

```sql
select *
from (

    select
        order_id

    from analytics.orders
    where order_id is not null
    group by order_id
    having count(*) > 1

) validation_errors
```

Not null test

Compiled SQL | Templated SQL

```
select *
from analytics.orders
where order_id is null
```

Running only data tests

To run data tests while excluding unit tests, use the `test_type` selector — this works across all engines (dbt Core and Fusion):

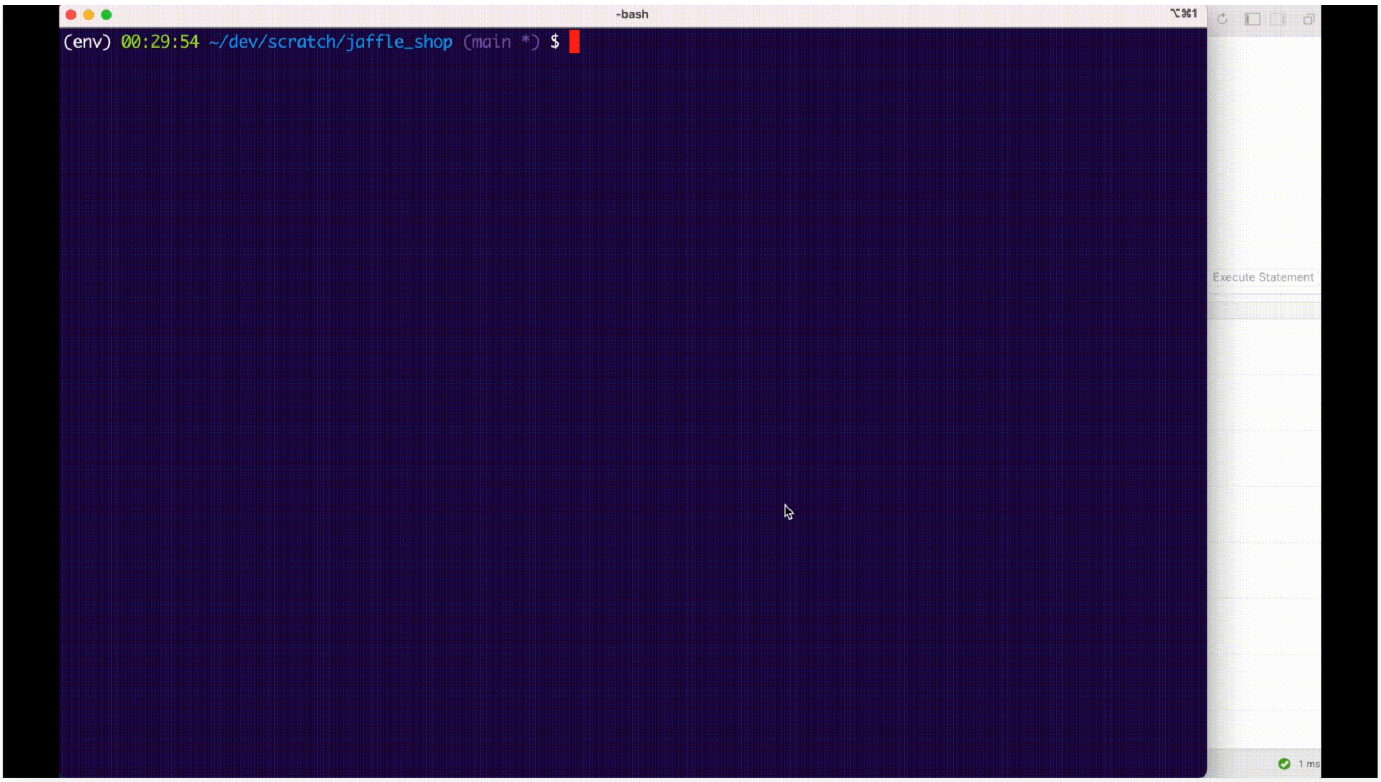
```
dbt test --select "test_type:data"
```

In dbt Core (v1.9+), you can also use `dbt test --resource-type test`. For more options, refer to test selection examples.

Storing data test failures

Normally, a data test query will calculate failures as part of its execution. If you set the optional `--store-failures` flag, the `store_failures`, or the `store_failures_as` configs, dbt will first save the results of a test query to a table in the database, and then query that table to calculate the number of failures.

This workflow allows you to query and examine failing records much more quickly in development:



Store test failures in the database for faster development-time debugging.

Note that, if you choose to store data test failures:

- Test result tables are created in a schema suffixed or named `dbt_test__audit`, by default. It is possible to change this value by setting a `schema` config. (For more details on schema naming, see using custom schemas.)
- A test's results will always **replace** previous failures for the same test.

New `data_tests:` syntax

Data tests were historically called "tests" in dbt as the only form of testing available. With the introduction of unit tests, the key was renamed from `tests:` to `data_tests:`.

dbt still supports `tests:` in your YAML configuration files for backward-compatibility purposes, and you might see it used throughout our documentation. However, you can't have a `tests` and a `data_tests` key associated with the same resource (for example, a single model) at the same time.

models/schema.yml

```
models:
  - name: orders
    columns:
      - name: order_id
        data_tests:
```

```
      - unique
      - not_null
```

dbt_project.yml

```
data_tests:
  +store_failures: true
```

FAQs

- What data tests are available for me to use in dbt?
- How do I test one model at a time?
- One of my tests failed, how can I debug it?
- What data tests should I add to my project?
- When should I run my data tests?
- Can I store my data tests in a directory other than the `tests` directory in my project?
- How do I run data tests on just my sources?
- Can I set test failure thresholds?
- Can I test the uniqueness of two columns?

Was this page helpful?

Yes No

Edit this page

*Last updated on **May 4, 2026***